

Project ShadowOS

A Background MicroVM & Execution Harness for Autonomous Agent Runtimes

STATUS	TARGET PLATFORMS	INTEGRATIONS	VERSION & DATE
Draft / Review	Linux (KVM), macOS (Apple VMM)	Claude Code, Antigravity, MCP	v1.0.0 · September 2026

1. Executive Summary & Vision

Modern autonomous developer agents (e.g., Claude Code, Google Antigravity IDE, SWE-agent) are fundamentally throttled by execution risks on bare-metal systems. Running commands on the host machine requires constant human prompts to prevent file destruction, credential leakage, or dependency corruption. Conversely, fully autonomous modes (`--dangerously-skip-permissions`) pose severe security hazards.

ShadowOS is an ultra-lightweight, hardware-isolated secondary virtual operating system that runs seamlessly in the background. It provides autonomous coding agents with an unconstrained, disposable sandbox powered by MicroVM virtualization, ephemeral Copy-on-Write storage, and a mediated Model Context Protocol (MCP) host promotion bridge.

Core Value Proposition

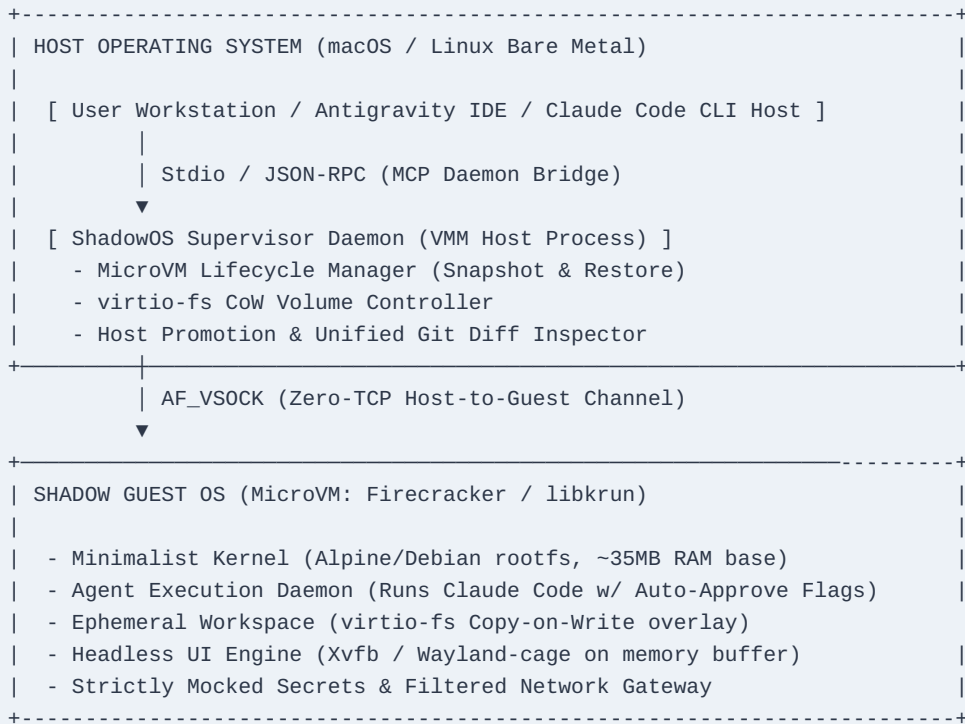
Safe 100% Autonomous Execution: Empowers agents to run builds, tests, browser interactions, and destructive CLI commands non-stop without human confirmation prompts, while strictly guaranteeing host security and zero RAM bloat (<250MB active overhead).

2. Problem Statement & Key Challenges

- **Permission Fatigue & Agent Stalls:** On bare-metal machines, agents prompt users before executing commands. If the user steps away, the agent halts, stretching minutes-long tasks into hours of idle wait.
- **System Pollution & Secret Exposure:** Autonomous scripts execute with the user's ambient privileges, exposing sensitive dotfiles (`~/ .ssh`, `~/ .aws`, environment variables) and polluting global runtimes (Python, Node, Docker daemons).
- **Slow and Fragile Error Recovery:** Fixing an agent's hallucinated migration or broken package tree requires manual `git checkout`, dependency re-installations, or manual container wipes.

3. System Architecture

The architecture isolates the unvetted execution layer inside a stripped-down guest Linux kernel while leaving the primary host environment intact.



4. Functional Requirements (FR)

ID	Feature Name	Priority	Technical Specification
FR-01	Sub-Second MicroVM Provisioning	P0	Spawns headless Linux MicroVMs via Firecracker (KVM) or libkrun (Apple VMM) in <150ms with a base memory footprint <150MB.
FR-02	Copy-on-Write Filesystem Bridge	P0	Mounts the host repository into the guest via virtio-fs with an ephemeral upper layer. Host files remain strictly read-only until user promotion.
FR-03	Zero-Prompt Agent Execution	P0	Natively executes agents (e.g. <code>claude --dangerously-skip-permissions</code>) without requiring interactive approval prompts.
FR-04	Host Promotion & Diff Review	P0	Generates unified git patches and verification summaries from the guest CoW layer, allowing single-click staging back to the host.
FR-05	Sub-100ms State Rollback	P1	Provides snapshot/restore checkpoints of the guest RAM and storage overlay before risky tasks to revert hallucinations instantly.
FR-06	Headless Virtual Display & Browser	P1	Integrates an in-memory Xvfb framebuffer and headless Chromium to run browser-use agents without stealing desktop focus.
FR-07	Antigravity IDE & MCP Integration	P1	Provides a Model Context Protocol (MCP) server that exports shadow execution primitives (<code>run_sandboxed_cmd</code> , <code>inspect_virtual_dom</code>).

5. Non-Functional Requirements (NFR)

- **Memory Footprint:** Base idle RAM consumption must not exceed 200MB. Peak RAM under active build workloads capped at 2.5GB.

- **Security Boundary:** Hardware-enforced virtualization boundary (Ring -1). No host environmental variables or unmounted directories accessible inside the guest.
- **I/O Performance:** File read/write operations across `virtio-fs` must maintain at least 85% of native NVMe throughput. In-memory builds (using `/tmp` RAM-disk) must exceed host physical disk speeds.
- **Cross-Platform Compatibility:** Support macOS 14+ (Apple Silicon via `Virtualization.framework`) and modern Linux distributions (Ubuntu 22.04+, Fedora 38+ with KVM enabled).

6. Integration Specifications

A. Claude Code Integration

The host CLI wrapper intercepts `claude` calls, transparently mounts the target directory into the ShadowOS guest, injects synthetic session credentials, and triggers execution with auto-approval flags enabled:

```
$ shadow-cli run claude --prompt "Migrate auth service to JWT and verify tests"
[*] Booted ShadowOS MicroVM (guest_id: vm_01jk98, boot: 112ms, ram: 128MB)
[*] Mounted ./project via virtio-fs (CoW layer active)
[*] Running unattended: claude -p "..." --dangerously-skip-permissions
[✓] 14 tests passed, 3 files modified.
[?] View diff and promote to host workspace? [Y/n]
```

B. Google Antigravity IDE Integration

ShadowOS exposes an MCP server running on the host over `stdio`. The Antigravity IDE targets this endpoint for sandboxed tasks, feeding artifacts (test runs, Playwright recordings, and terminal outputs) directly into the Antigravity Manager View.

7. Success Metrics (KPIs)

- **Wall-Clock Task Completion:** >3.5x reduction in end-to-end task time for unattended agent coding jobs (eliminating human idle pauses).
- **Zero Host Incidents:** 0 instances of host filesystem corruption, unauthorized port bindings, or credential leaks during test suites.
- **Rollback Latency:** <100ms mean time to reset environment state after an agent execution failure.

8. Release Roadmap

- **Phase 1 (M1 - Core Engine):** Headless MicroVM runner with Firecracker/libkrun, `virtio-fs` mounting, and basic bash execution via `AF_VSOCK`.
- **Phase 2 (M2 - Harness Tooling):** Claude Code wrapper, automated CoW promotion git-diff UI, and snapshot rollback checkpoints.
- **Phase 3 (M3 - Ecosystem Expansion):** Antigravity MCP server, headless virtual display (Xvfb) for browser agents, and parallel multi-agent swarming.